

Practical Work No. 16

Linear Lists

1. Introduction

A linear list (LL) is the computer representation of a finite set of variable and possibly zero size elements of type T. Such a set is ordered.

Mathematically, an LL can be presented by enumerating its elements in order.

For the stack data structure, additions (push), deletions (pop), and searches (top) are performed relative to the top. The stack data structure is said to be a single access point structure.

For the queue data structure, additions (enqueue) are performed at the rear, while deletions (dequeue) and searches (front) are performed at the front. The queue data structure is said to be a two access points structure.

For the LL data structure, additions, deletions, and searches are not systematically performed relative to either the head or the tail.

2. Operations on the LL Data Structure

We distinguish between atomic (or elementary) operations and advanced operations.

2.1 Atomic Operations

- **Create_liste:** Creates an empty linear list.
- **Liste_vide:** Checks whether the linear list is empty.
- **Add (insertion operation):**
 - At the head: before the first element.
 - At the tail: after the last element.
 - Somewhere in the middle.
- **Remove an element from the list:**
 - First element.
 - Last element.
 - Somewhere in the middle.
- **Search:** Checks whether an element belongs to a linear list. The starting point can be provided as a parameter.
- **Visit:** Visits all elements of the LL data structure, performing a given action (parameter) on each visited element. This action is known as traversal.

2.2 Advanced Operations

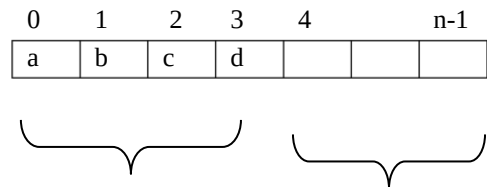
- **Reverse :** Inverts a linear list $ll=(a, b, c, d)$ $ll_{inv}=(d, c, b, a)$
- **RemoveAll:** Removes all elements from a given list.
- **Concat:** Concatenates two given lists. $ll_1=(a, b, c, d)$ $ll_2=(x, y, z, w, k)$

3. Physical Representation

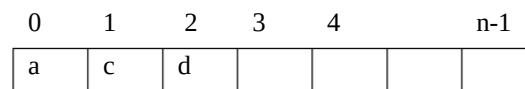
We distinguish: contiguous representation and linked representation.

3.1 Contiguous Representation

$ll=(a, b, c, d)$ Abstract representation

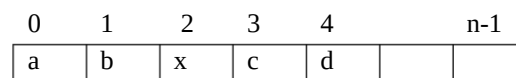


Remove: It requires left shifts to reclaim the now available position. Example: remove b.



Search: Search in an array, we use known algorithms such as sequential search or binary search.

Adjunction: It requires right shifts. Example: add x after b.

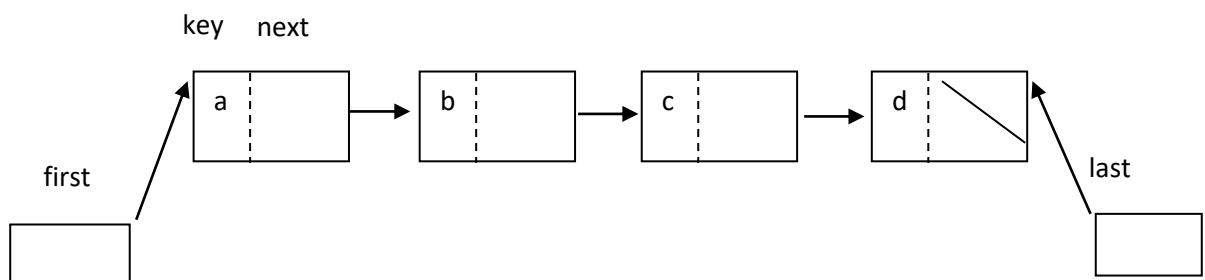


Visit: Sweep through all the elements of an array in the filled portion.

3.2 Linked Representation

$ll=(a, b, c, d)$

variante 1 :

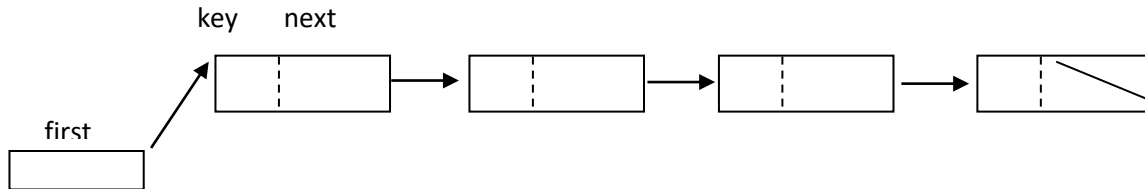


Note: Insertion in a linear list (LL) realized using linked representation does not require shifts, unlike contiguous representation.

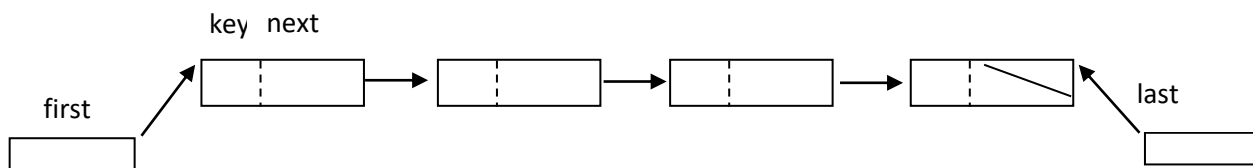
It can be said that the contiguous representation of the linear list data structure (SDLL) is not recommended due to the shifts involved in the two fundamental operations: adding and removing, especially for linear lists of considerable size.

4. Variants of the Linear List Data Structure We distinguish:

4.1 Unidirectional Linear List with a single entry point

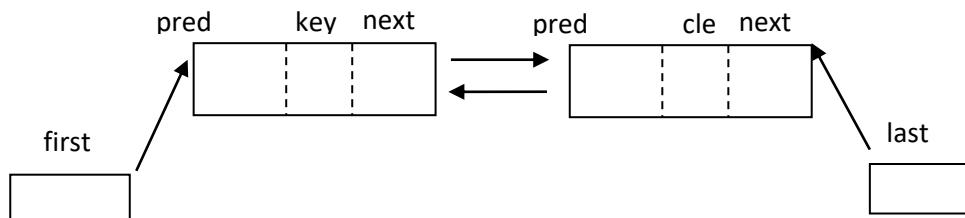


4.2 Unidirectional Linear List with two entry points



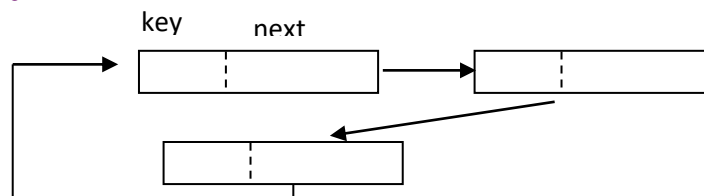
For both variants (1) and (2), the linear list is unidirectional. From a given element, one can move to its successor. This is made possible by the chaining field: in variant (1), the first element is privileged (direct access), while in variant (2), both the first and last elements are privileged (direct access).

4.3 Liste linéaire bidirectionnelle avec 2 points d'entrée



From a given element, one can move either to its successor or to its predecessor.

4.4 Circular Linear List



The notions of first and last do not exist in a circular list. A circular list has only an arbitrary entry point.

5 Singly Linked List with Two Entry Points

5.1 A singly linked list with a head and a tail

/*Physical Representation */

```
struct nœud {
    int key ;

    struct nœud * next ;
};

struct liste {
    struct nœud* first ;

    struct nœud * last ;
};
```

/* Creation of a linear list */

Solution 1: As a procedure	Solution 2 : As a function
<pre>void create_liste (struct liste * ll) { ll->first=NULL ; ll->last=NULL ; }</pre>	<pre>struct liste * creat_liste (void) { struct liste *ll ; ll=(struct liste*)malloc (size of(struct liste)) ; ll->first=NULL; ll->last=NULL; return ll; }</pre>

/*Test the emptiness of a linear list*/

Solution 1	Solution 2
<pre>unsigned liste_vide (struct liste *ll) { return(ll->first==NULL) ; }</pre>	<pre>unsigned liste_vide (struct liste*ll) { return((ll->first==NULL)&&(ll->last==NULL)); }</pre>

/*Addition or insertion process*/

/*insertion: insert after a referenced element

insert before a referenced element

insert before first

insert after last*/

/*insertion after a referenced element*/

void insert_after(int info, struct node *p) {

 struct node *q;

 q = (struct node*) malloc(sizeof(struct node));

 q->key = info;

 q->next = p->next;

/*update the successor of p*/

 p->next = q;

}

/*insertion before a referenced element*/

void insert_before(int info, struct node *p) {

/* the problem: the list is unidirectional from p, we cannot access its predecessor */

 struct node *q;

 q = (struct node*) malloc(sizeof(struct node));

 *q = *p;

/*in C, assignment is defined for variables with a struct-based type:

// q->key = p->key, q->next = p->next*/

/*update the memory referenced by p*/

 p->key = info;

 p->next = q;

}

```
/* Search in a linear list*/
```

```
/* A search problem with two outcomes: success (non-null reference) and failure (null reference)*/
```

```
struct node *search(int info, struct node *p) {
```

```
/* It returns a non-null reference if info belongs to the list starting from the element referenced by p,
   otherwise it returns a null reference */
```

```
/* Common point: sequential search algorithm in an array, in the context of a linear list */
```

```
while (p && (p->key != info))
```

```
/* the order of the two sub-expressions is significant */
```

```
p = p->next;
```

```
/* move to the next element */
```

```
/* at the end of the loop: !p || p->key == info
```

```
failure: !p => p == NULL
```

```
success: p->key == info */
```

```
return p;
```

```
}
```

5.2 Traversal operation

Visit all the elements of a linear list while performing a given operation.

Reminder about parameter types in C:

Non-functional parameter: it is a parameter that stores a value (ordinary or an address).

Functional parameter: it stores a behavior (or an operation) represented by a subprogram. Subprograms that accept functional parameters are called higher-order subprograms.

What are the necessary parameters for the traversal operation?

Non-functional parameter:

It indicates the starting point to traverse the list: `struct node *p;`

Functional parameter:

It indicates the operation to perform on the visited nodes: `void(*oper)(struct node*)`

`oper` is a pointer to a procedure requiring a parameter of type `struct node*`

Do not confuse!

```
void* oper(struct node*);
```

This defines a function named `oper` that returns a pointer to anything and takes a parameter of type `struct node*`.

```
void traverse(struct node *p, void (*oper)(struct node*)) {
    while (p) {
        (*oper)(p); /* apply to the element pointed to by p the operation provided by oper */
        p = p->next; /* move to the next element */
    }
}
```

Using the traverse procedure:

1- Actual parameters:

```
struct node *starting_point;
```

```
void display(struct node *q) {
```

```
    printf("%u\n", q->key);
```

```
}
```

```
/* activation of traverse */
```

```
traverse(starting_point, display);
```

In C, the name of a subprogram is considered as a pointer.

Any reference to `(*oper)` within the body of the `traverse` subprogram is a reference to the corresponding actual parameter: this is the principle of indirection.

2- Processing: increment each visited element:

The operation consists of incrementing all the visited elements

```
struct node *starting_point;
```

```
void increment(struct node *q) {
```

```
    q->key++;
```

```
}
```

```
/* activation */
```

```
traverse(starting_point, increment);
```

5.3. Doubly linked list with two entry points

```
struct node {
```

```
    struct node *prev;
```

```
    int key;
```

```
    struct node *next;
```

```
};
```

```
struct list {
```

```
    struct node *first;
```

```
    struct node *last; };
```

6. Application Exercises

Exercise 1: Reversing a linked list

Write a procedure that reverses a linked list Ls into a list Ls_Inv.

Exercise 2: Insertion into a linked list

Assume the input is a linked list sorted in ascending order of integers. Write a procedure that inserts an integer in the correct position so that the list remains sorted after the insertion.

Exercise 4: Functional parameter

a) Write a C subprogram to display the elements of a linear list that satisfy a given selection criterion. Such a criterion can express any condition.

b) Example of using the subprogram from (a)

Exercise 6: Merge Sort

1- Propose a method that concatenates two lists and returns a list. For example, if we have the lists (3,7,2) and (5,4), we get the list (3,7,2,5,4).

2- Then propose a method that splits a list into two. Given an input list, every other element goes into the first list and every other element goes into the second list.

3- Perform a merge action of lists assumed to be sorted in ascending order into a third sorted list.

4- Using any of the previous three actions, create a merge sort action that, given an input list, returns a sorted list.

```

struct liste * concatenerListes (struct liste *L1, struct liste *L2); /* to be implemented */
void Separer (struct liste *L, struct liste *L1, struct liste *L2); /* to be implemented */
struct liste * Fusion (struct liste *L1, struct liste *L2); /* to be implemented */
struct liste * TriFusion (struct liste *L) {
    struct liste *L1 = (struct liste*) malloc (sizeof(struct liste));
    struct liste *L2 = (struct liste*) malloc (sizeof(struct liste));
    struct liste *L1b;
    struct liste *L2b;
    assert (! liste_vide(L));
    if (L->first->next) { /* contains more than one element */
        Separer (L, L1, L2);
        L1b = TriFusion (L1);
        L2b = TriFusion (L2);
        return Fusion (L1b, L2b);
    }
    else {
        create_liste (L1);
        insert_before_first (L->first->key, L1);
        return L1;
    }
}

```